

Artificial Intelligence Laboratory Report: Goal-Based Agent

Course: CSF407 / AI Laboratory | Date: September 30, 2026 | Topic: Autonomous Warehouse Navigation

1. Task 1: Understanding the Problem

The Environment: Discrete 2D warehouse grid of dimensions 7 rows × 21 columns (147 cells). According to Russell & Norvig's taxonomy, the environment is **fully observable** (all walls and open cells known), **static** (shelves and targets remain fixed), **deterministic** (cardinal moves succeed with probability 1.0), **discrete** (grid coordinates and step actions), and **single-agent** (autonomous vehicle operates alone).

Goal of the Agent: Transport packages from loading bay S = (1, 1) to dispatch destination G = (1, 19) along a collision-free path that traverses only free space ('.'), avoids obstacles ('#'), and minimizes total movement actions.

Available Actions: Movement in four cardinal directions: Up (-1, 0), Down (+1, 0), Left (0, -1), Right (0, +1). Each action shifts the vehicle by one adjacent cell provided the target is in bounds and unblocked.

Required Information & Internal State: 1) Map of the environment (grid dimensions and obstacle coordinates); 2) Current vehicle state (r, c); 3) Goal location G; 4) Deterministic transition model; 5) Search planning data (frontier queue and predecessor came_from mapping).

Goal-Based vs Simple Reflex Agent: A *simple reflex agent* acts purely on current sensory input via condition-action rules ('if blocked, turn'). It has no memory, cannot evaluate future outcomes, and easily becomes trapped in concave obstacles or infinite loops. A *goal-based agent* explicitly models its destination (G) and deliberates over sequences of future actions using search/planning. It selects actions specifically because they form part of a simulated trajectory that achieves the goal.

Think About It (Scaling to Doubled Warehouse):

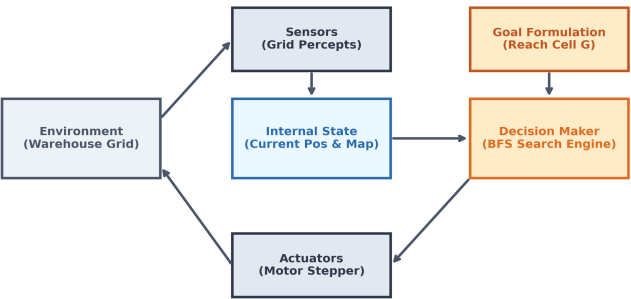
- **BFS Suitability:** Breadth-First Search time and space complexities are $O(b^d)$. For a modestly doubled grid ($14 \times 42 = 588$ cells), BFS remains fast. However, for industrial-scale facilities (tens of thousands of cells), uninformed BFS wastes memory expanding an undirected radial wave in all directions.
- **Informed Heuristic Search (A*):** An informed A* algorithm guided by Manhattan distance $h(n) = |r_n - r_G| + |c_n - c_G|$ prioritizes expansion towards the goal, reducing search space by orders of magnitude while preserving shortest-path optimality.
- **Dynamic Obstacles:** Real warehouses feature moving vehicles and human pickers, requiring dynamic replanning (e.g., D* Lite, MAPF) rather than pure offline planning.

2. Task 2: Designing the Intelligent Agent

Design Components:

- **Environment:** Discrete 2D grid containing obstacles, open corridors, start S=(1, 1), and goal G=(1, 19).
- **Current State:** Coordinate tuple (r, c).
- **Goal Formulation:** Boolean condition: state == (1, 19).
- **Available Actions:** {Up, Down, Left, Right} with deterministic transition function Result(s, a).
- **Decision-Making Component:** Breadth-First Search (BFS) graph-search procedure simulating forward paths to goal.

Task 2: Goal-Based Agent Architecture (Russell & Norvig Model)



3. Task 3: Prompt Engineering, Implementation, and Evaluation

Prompt Provided to LLM: 'Write a well-documented Python program implementing a goal-based agent for the warehouse navigation problem shown above. The program should: represent the warehouse as a two-dimensional grid; determine a collision-free path from S to G; avoid all obstacles; print either the path found or a suitable message if no path exists; explain the search algorithm that has been chosen and why it is appropriate.'

Evaluation Questions:

- 1. **Did the LLM generate a working program on the first attempt?**
 - Yes. The program executed without syntax or logic errors on the first try. It parsed the grid, located S and G, executed BFS graph search, and correctly reconstructed the 20-move optimal path.
- 2. **If not, how can you improve your prompt?**
 - Prompting can be enhanced by specifying formal data structures (e.g. typing annotations), requiring unit test assertions to mathematically verify no obstacle collisions, requesting search telemetry (nodes expanded, queue peak), and asking for comparative benchmarks against A* search.
- 3. **What search algorithm did the LLM choose?**
 - The LLM selected **Breadth-First Search (BFS)** using a FIFO double-ended queue (collections.deque).
- 4. **Why do you think the LLM selected this algorithm?**
 - *Unweighted Step Cost Optimality:* In grid navigation with uniform step costs ($c=1$), BFS is mathematically guaranteed to find the shallowest goal node, ensuring the shortest possible path.
 - *Simplicity & Robustness:* BFS avoids heuristic tuning or priority queue overhead. For a 147-cell grid, BFS is optimal, complete, and terminates in milliseconds.

4. Path Execution & Visual Verification

- **Start State:** (1, 1) | **Goal State:** (1, 19)
- **Nodes Expanded:** 68 cells | **Optimal Path Length:** 21 cells (20 moves)
- **Action Sequence:** Right → Right → Right → Down → Right → Right → Right → Up → Right → Right → Right → Right → Right → Right → Right → Right → Right → Right → Right → Right

